Tutorial 4

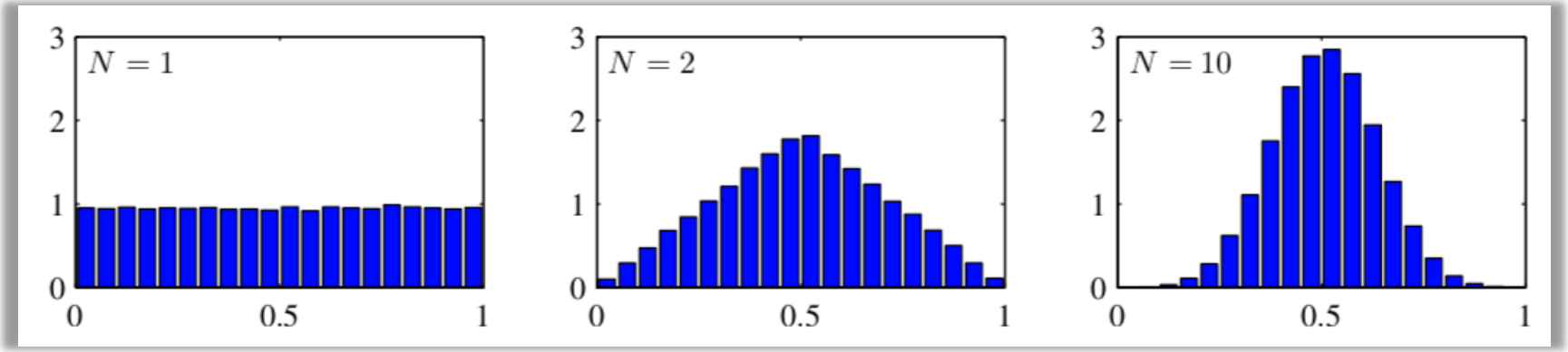
Gaussian Distribution, Bias-Variance visualization,
and Gradient decent

The Gaussian Distribution

- For a D-dimensional vector x, the multivariate Gaussian distribution takes the form:

$$\mathcal{N}(x|\mu,\Sigma) = \frac{1}{(2\pi)^{\frac{D}{2}}|\Sigma|^{\frac{1}{2}}}\exp\left\{-\frac{1}{2}(x-\mu)^{\top}\Sigma^{-1}(x-\mu)\right\}$$

- Motivations:
  - Maximum of the entropy
  - Central limit theorem



The Gaussian Distribution: Properties

- The law is a function of the Mahalanobis distance from x to μ:

$$\Delta^2 = (x - \mu)^\top \Sigma^{-1} (x - \mu)$$

- The expectation of x under the Gaussian distribution is:

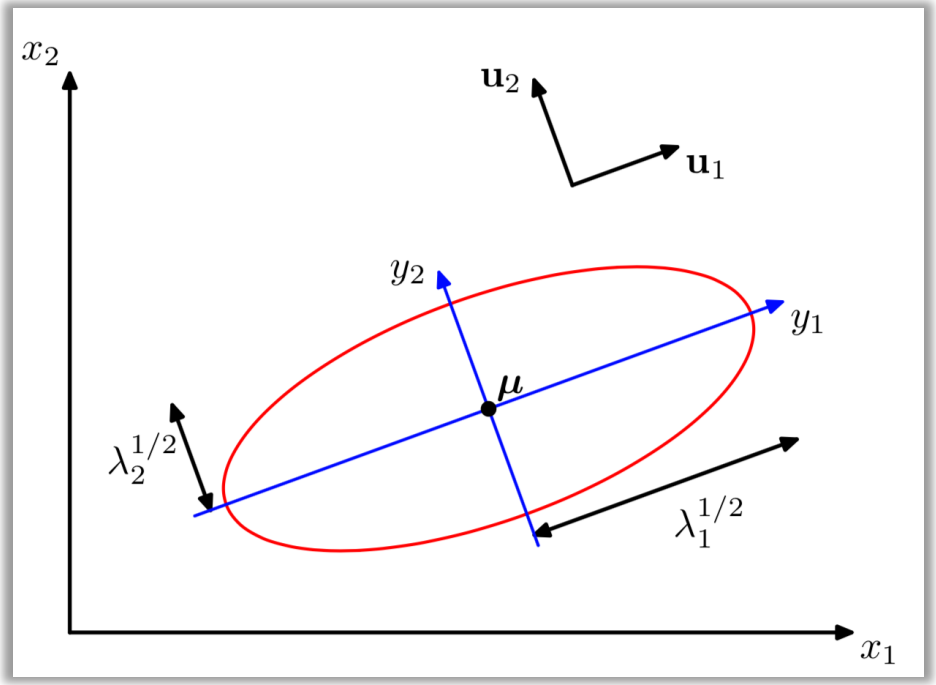
$$\mathbb{E}[x] = \mu$$

- The covariance matrix of x is:

$$\mathrm{cov}[x] = \Sigma$$

The Gaussian Distribution: Properties

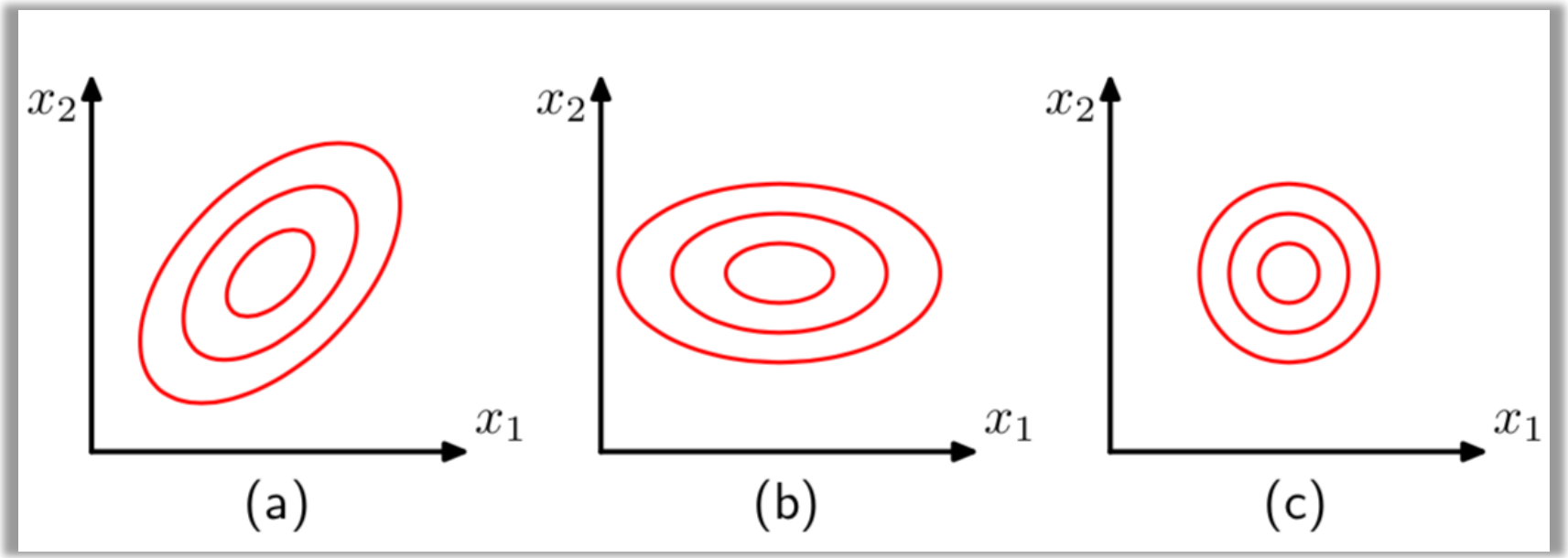
- The law (quadratic function) is constant on elliptical surfaces:



- λ_i are the eigenvalues of Σ
- u_i are the associated eigenvectors

The Gaussian Distribution: more examples

- Contours of constant probability density:



a) general form
b) diagonal
c) proportional to the identity matrix

Conditional Law

- Given a Gaussian distribution $\mathcal{N}(x|\mu,\Sigma)$ with

$$x = (x_a, x_b)^\top, \qquad \mu = (\mu_a, \mu_b)^\top$$

$$\Sigma = \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix} = \begin{pmatrix} \Lambda_{aa} & \Lambda_{ab} \\ \Lambda_{ba} & \Lambda_{bb} \end{pmatrix}^{-1}$$

$$-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu) =$$

$$-\frac{1}{2}(x_a-\mu_a)^\top \Lambda_{aa}(x_a-\mu_a) - \frac{1}{2}(x_a-\mu_a)^\top \Lambda_{ab}(x_b-\mu_b)$$

$$-\frac{1}{2}(x_b-\mu_b)^\top \Lambda_{ba}(x_a-\mu_a) - \frac{1}{2}(x_b-\mu_b)^\top \Lambda_{bb}(x_b-\mu_b)$$

- What's the conditional distribution $p(x_a|x_b)$?

Conditional Law

- What's the conditional distribution $p(x_a|x_b)$?

$$-\frac{1}{2}(x-\mu)^\top\Sigma^{-1}(x-\mu) = -\frac{1}{2}x^\top\Sigma^{-1}x + x^\top\Sigma^{-1}\mu + \text{const}$$

  - $\Sigma_{a|b}^{-1} = \Lambda_{aa}$
  - $\Sigma_{a|b}^{-1}\mu_{a|b} = \Lambda_{aa}\mu_a - \Lambda_{ab}(x_b - \mu_b)$

- Using the definition:

$$\Sigma = \begin{pmatrix}\Sigma_{aa} & \Sigma_{ab}\\ \Sigma_{ba} & \Sigma_{bb}\end{pmatrix} = \begin{pmatrix}\Lambda_{aa} & \Lambda_{ab}\\ \Lambda_{ba} & \Lambda_{bb}\end{pmatrix}^{-1}$$

  - $\Lambda_{aa} = \left(\Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}\right)^{-1}$
  - $\Lambda_{ab} = -\left(\Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}\right)^{-1}\Sigma_{ab}\Sigma_{bb}^{-1}$

Inverse partition identity:

$$\begin{pmatrix}A & B\\ C & D\end{pmatrix}^{-1} = \begin{pmatrix}M & -MBD^{-1}\\ -D^{-1}CM & D^{-1} + D^{-1}CMBD^{-1}\end{pmatrix} \quad M = (A - BD^{-1}C)^{-1}$$

Conditional Law

- The conditional distribution $p(x_a|x_b)$ is a Gaussian with:

$$\mu_{a|b} = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(x_b - \mu_b)$$

$$\Sigma_{a|b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$$

- The form using precision matrix:

$$\mu_{a|b} = \mu_a + \Lambda_{aa}\Lambda_{ab}^{-1}(x_b - \mu_b)$$

$$\Lambda_{a|b} = \Lambda_{aa}$$

Marginal Law

- The marginal distribution is given by:

$$p(x_a) = \int p(x_a, x_b) dx_b$$

- Picking out those terms that involve x_b, we have

$$-\frac{1}{2} x_b^\top \Lambda_{bb} x_b + x_b^\top m = -\frac{1}{2}\left(x_b - \Lambda_{bb}^{-1} m\right)^\top \Lambda_{bb}\left(x_b - \Lambda_{bb}^{-1} m\right) + \frac{1}{2} m^\top \Lambda_{bb}^{-1} m$$

$$m = \Lambda_{bb}\mu_b - \Lambda_{ba}(x_a - \mu_a)$$

- Integrate over x_b (unnormalized Gaussian)

$$\int \exp\left\{-\frac{1}{2}\left(x_b - \Lambda_{bb}^{-1} m\right)^\top \Lambda_{bb}\left(x_b - \Lambda_{bb}^{-1} m\right)\right\} dx_b$$

✓ The integral is equal to the normalization term

Marginal Law

- After integrating over x_b, we pick out the remaining terms:

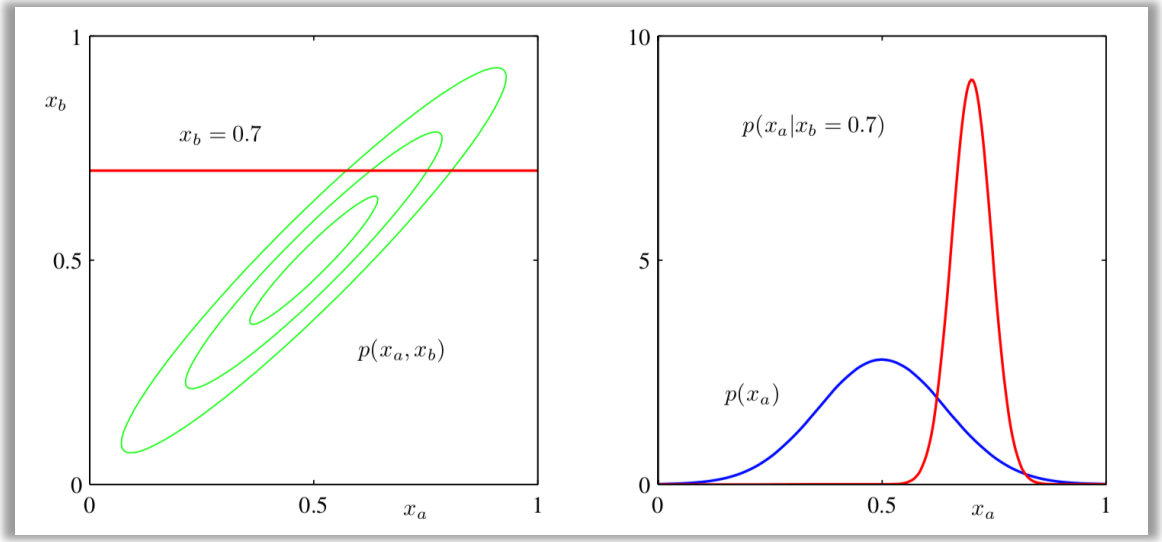
$$-\frac{1}{2}x_a^\top \Lambda_{aa} x_a + x_a^\top(\Lambda_{aa}\mu_a + \Lambda_{ab}\mu_b) + \frac{1}{2}m^\top \Lambda_{bb}^{-1} m + \text{const}$$

$$m = \Lambda_{bb}\mu_b - \Lambda_{ba}(x_a - \mu_a)$$

- The marginal distribution is a Gaussian with

$$\mathbb{E}[x_a] = \mu_a \qquad \text{cov}[x_a] = \Sigma_{aa}$$

Short Summary



$$\Sigma = \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix} = \begin{pmatrix} \Lambda_{aa} & \Lambda_{ab} \\ \Lambda_{ba} & \Lambda_{bb} \end{pmatrix}^{-1}$$

- Conditional distribution:

$$p(x_a|x_b) = \mathcal{N}\left(x_a|\mu_{a|b}, \Lambda_{aa}^{-1}\right)$$

$$\mu_{a|b} = \mu_a - \Lambda_{aa}^{-1}\Lambda_{ab}(x_b - \mu_b)$$

- Marginal distribution:

$$p(x_a) = \mathcal{N}(x_a|\mu_a, \Sigma_{aa})$$

Bayes' theorem for Gaussian variables

- Setup:

$$p(x) = \mathcal{N}(x|\mu, \Lambda^{-1})$$

$$p(y|x) = \mathcal{N}(y|\mathrm{A}x + b, \mathrm{L}^{-1})$$

- What's the marginal distribution $p(y)$ and conditional distribution $p(x|y)$?
  - ✓ How about first compute $p(z)$, where $z = (x, y)^{\mathsf{T}}$
  - ✓ $p(z)$ is a Gaussian distribution, consider the log of the joint distribution

$$\begin{aligned}\ln p(z) &= \ln p(x) + \ln p(y|x) \\ &= -\frac{1}{2}(x-\mu)^{\mathsf{T}}\Lambda(x-\mu) \\ &\quad -\frac{1}{2}(y-\mathrm{A}x+b)^{\mathsf{T}}\mathrm{L}(y-\mathrm{A}x+b) + \text{const}\end{aligned}$$

Bayes' theorem for Gaussian variables

- The same trick (consider the second order terms), we get

$$\mathbb{E}[z] = \begin{pmatrix} \mu \\ \mathrm{A}\mu + b \end{pmatrix}$$

$$\mathrm{cov}[z] = \begin{pmatrix} \Lambda^{-1} & \Lambda^{-1}\mathrm{A} \\ \mathrm{A}\Lambda^{-1} & L^{-1} + \mathrm{A}\Lambda^{-1}\mathrm{A} \end{pmatrix}$$

- We can then get $p(y)$ and $p(x|y)$ by marginal and conditional laws!

Maximum likelihood for the Gaussian

- Assume we have $\mathrm{X} = (x_1, \ldots, x_N)^\top$ in which the observation $\{x_n\}$ are assumed to be drawn independently from a multivariate Gaussian, the log likelihood function is given by

$$\ln p(\mathrm{X}|\mu, \Sigma) = -\frac{ND}{2}\ln 2\pi - \frac{N}{2}\ln|\Sigma| - \frac{1}{2}\sum_{n=1}^{N}(x_n - \mu)^\top \Sigma^{-1}(x_n - \mu)$$

- Setting the derivative to zero, we obtain the solution for the maximum likelihood estimator:

$$\frac{\partial}{\partial \mu}\ln p(\mathrm{X}|\mu, \Sigma) = \sum_{n=1}^{N}\Sigma^{-1}(x_n - \mu) = 0$$

$$\mu_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}x_n \qquad \Sigma_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}(x_n - \mu_{\mathrm{ML}})(x_n - \mu_{\mathrm{ML}})^\top$$

Maximum likelihood for the Gaussian

- The empirical mean is unbiased in the sense

$$\mathbb{E}[\mu_{\mathrm{ML}}] = \mu$$

- However, the maximum likelihood estimate for the covariance has an expectation that is less that the true value:

$$\mathbb{E}[\Sigma_{\mathrm{ML}}] = \frac{N-1}{N}\Sigma$$

  - ✓ We can correct it by multiplying Σ_{ML} by the factor $\frac{N}{N-1}$

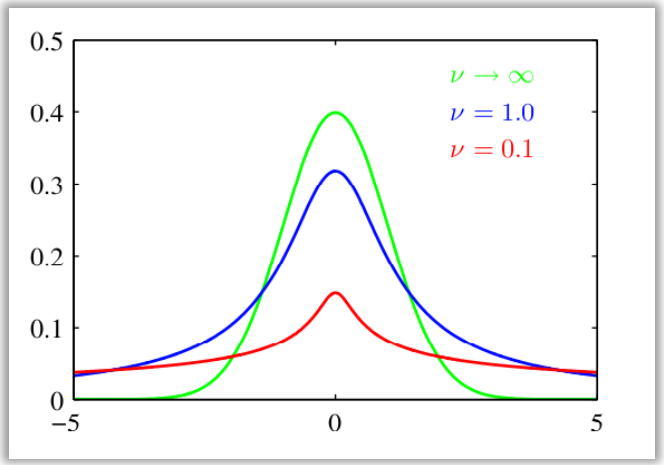
Conjugate prior for the Gaussian

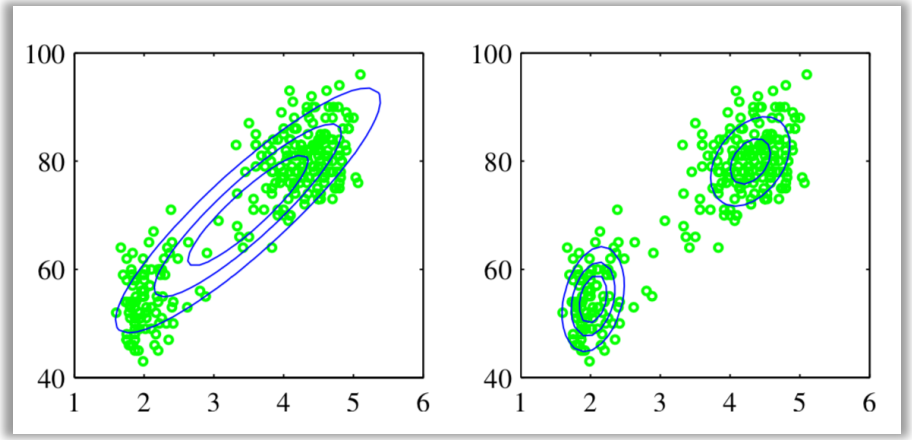
- The maximum likelihood framework only gives point estimates for the parameters, we would like to have uncertainty estimation (confidence interval) for the estimation
  - ✓ Introducing prior distributions over the parameters of the Gaussian

- We would like the posterior $p(\theta|D) \propto p(\theta)p(D|\theta)$ has the same form as the prior (Conjugate prior!)
  - ✓ The conjugate prior for μ is a Gaussian
  - ✓ The conjugate prior for precision Λ is a Gamma distribution

The Gaussian Distribution: limitations

- A lot of parameters to estimate $D + \frac{D^2+D}{2}$: structured approximation (e.g., diagonal variance matrix)
- Maximum likelihood estimators are not robust to outliers: Student's t-distribution (bottom left)
- Not able to describe periodic data: von Mises distribution
- Unimodel distribution: Mixture of Gaussian (bottom right)





The Gaussian Distribution: frontiers

- ❑ Gaussian Process
- ❑ Bayesian Neural Networks
- ❑ Generative modeling (Variational Autoencoder)
- ❑ ……

Bias-Variance Decomposition for the General Estimator

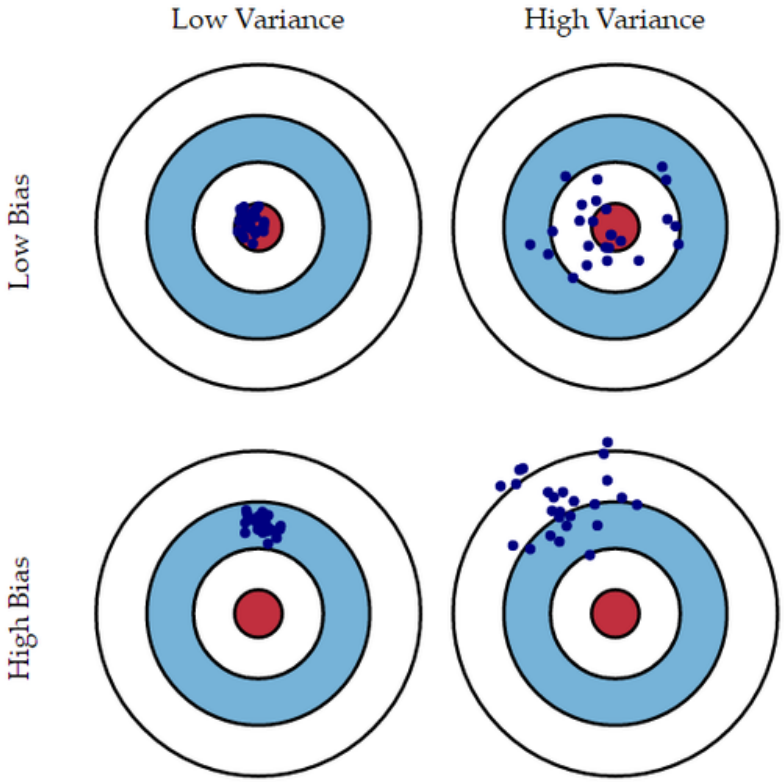
Bias-Variance Decomposition

$$\mathbb{E}_{\mathcal{D},\mathbf{x}}\left[\left|h(\mathbf{x};\mathcal{D}) - y_*(\mathbf{x})\right|^2\right] = \underbrace{\mathbb{E}_{\mathbf{x}}\left[\left|\mathbb{E}_{\mathcal{D}}\left[h(\mathbf{x};\mathcal{D}) \mid \mathbf{x}\right] - y_*(\mathbf{x})\right|^2\right]}_{\text{bias}} + \underbrace{\mathbb{E}_{\mathcal{D},\mathbf{x}}\left[\left|h(\mathbf{x};\mathcal{D}) - \mathbb{E}_{\mathcal{D}}\left[h(\mathbf{x};\mathcal{D}) \mid \mathbf{x}\right]\right|^2\right]}_{\text{variance}}.$$

- Bias: The squared error between the average estimator (averaged over dataset $\mathcal{D}$) and the best predictor $y_*(\mathbf{x}) = \mathbb{E}\left[t|\mathbf{x}\right]$, averaged over $\mathbf{x} \sim p_{\mathbf{x}}$.
- Variance: The variance of a single estimator $h(\mathbf{x};\mathcal{D})$ (whose randomness comes from $\mathcal{D}$).

Bias-Variance Decomposition: A Visualization

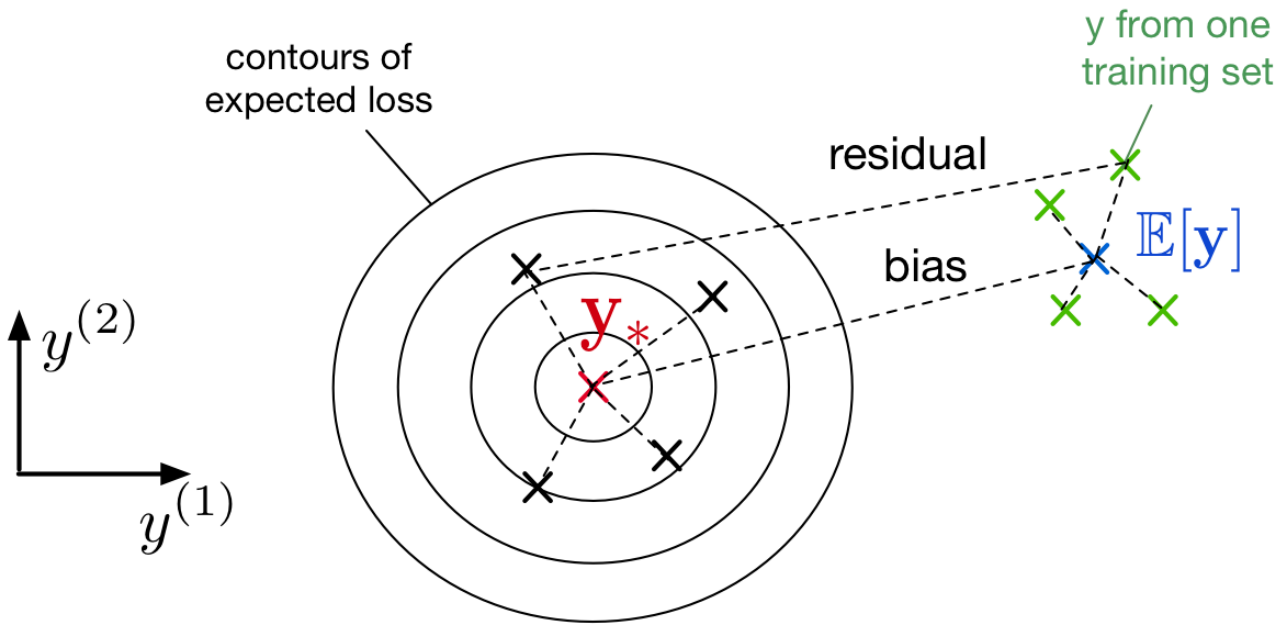
- Throwing darts = predictions for each draw of a dataset



- What doesn't this capture?
- We average over points $\mathbf{x}$ from the data distribution

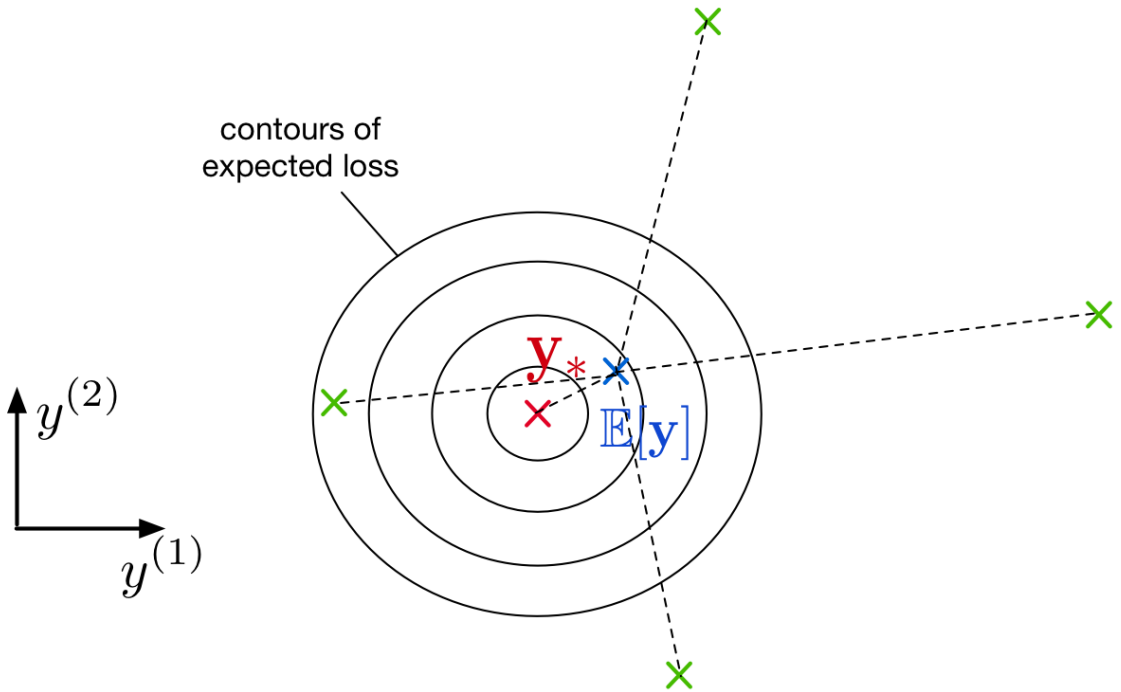
Bias-Variance Decomposition: Another Visualization

- We can visualize this decomposition in **output space**, where the axes correspond to predictions on the test examples.
- If we have an overly simple model (e.g. k-NN with large k), it might have
  - high bias (because it is too simplistic to capture the structure in the data)
  - low variance (because there is enough data to get a stable estimate of the decision boundary)



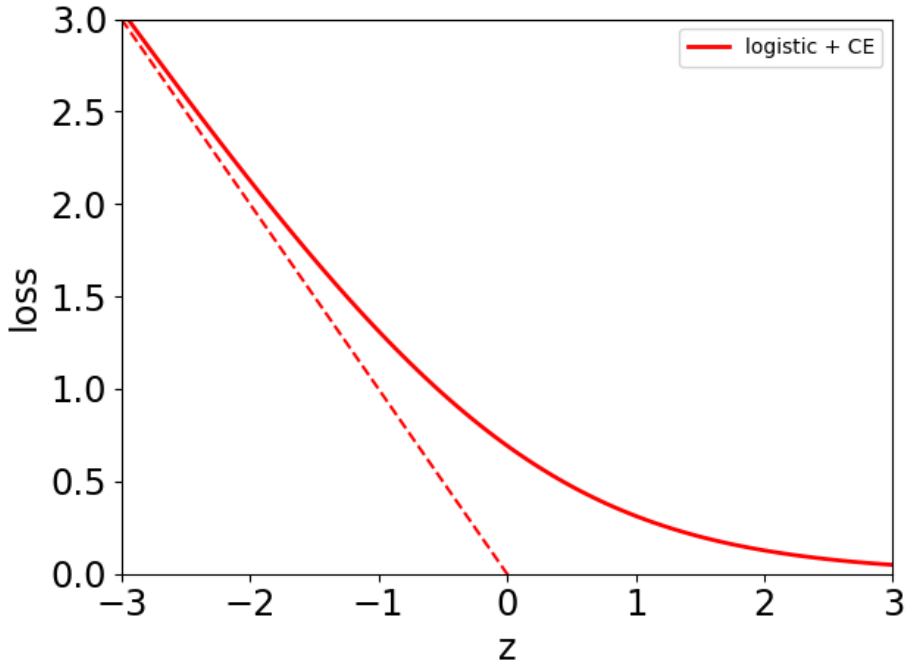
Bias-Variance Decomposition: Another Visualization

- If you have an overly complex model (e.g. k-NN with $k = 1$), it might have
  - low bias (since it learns all the relevant structure)
  - high variance (it fits the quirks of the data you happened to sample)



Logistic Regression:

$$z = \mathbf{w}^\top \mathbf{x} + b$$
$$y = \sigma(z)$$
$$= \frac{1}{1 + e^{-z}}$$
$$\mathcal{L}_{\text{CE}} = -t \log y - (1 - t) \log(1 - y)$$



Plot is for target $t = 1$.

Gradient Descent

- How do we minimize the cost $\mathcal{J}$ in this case? No direct solution.
  - Taking derivatives of $\mathcal{J}$ w.r.t. $\mathbf{w}$ and setting them to 0 doesn't have an explicit solution.
- Now let's see a second way to minimize the cost function which is more broadly applicable: gradient descent.
- Gradient descent is an iterative algorithm, which means we apply an update repeatedly until some criterion is met.
- We initialize the weights to something reasonable (e.g. all zeros) and repeatedly adjust them in the direction of steepest descent.

Gradient for Logistic Regression

Back to logistic regression:

$$\mathcal{L}_{\text{CE}}(y,t) = -\,t\log(y) - (1-t)\log(1-y)$$
$$y = 1/(1+e^{-z}) \quad \text{and} \quad z = \mathbf{w}^T\mathbf{x} + b$$

Therefore

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial w_j} = \frac{\partial \mathcal{L}_{\text{CE}}}{\partial y}\cdot\frac{\partial y}{\partial z}\cdot\frac{\partial z}{\partial w_j} = \left(-\frac{t}{y} + \frac{1-t}{1-y}\right)\cdot y(1-y)\cdot x_j$$
$$= (y-t)x_j$$

Exercise: Verify this!

Gradient descent (coordinatewise) update to find the weights of logistic regression:

$$w_j \leftarrow w_j - \alpha\frac{\partial \mathcal{J}}{\partial w_j}$$
$$= w_j - \frac{\alpha}{N}\sum_{i=1}^{N}(y^{(i)} - t^{(i)})\,x_j^{(i)}$$

Logistic Regression

Comparison of gradient descent updates:

- Linear regression (verify!):

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)}) \, \mathbf{x}^{(i)}$$

- Logistic regression:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)}) \, \mathbf{x}^{(i)}$$

- Not a coincidence! These are both examples of generalized linear models. But we won't go in further detail.
- Notice $\frac{1}{N}$ in front of sums due to averaged losses. This is why you need smaller learning rate when we optimize the sum of losses ($\alpha' = \alpha/N$).

Stochastic Gradient Descent

- So far, the cost function $\mathcal{J}$ has been the average loss over the training examples:

$$\mathcal{J}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N}\mathcal{L}^{(i)} = \frac{1}{N}\sum_{i=1}^{N}\mathcal{L}(y(\mathbf{x}^{(i)}, \boldsymbol{\theta}), t^{(i)}).$$

- By linearity,

$$\frac{\partial \mathcal{J}}{\partial \boldsymbol{\theta}} = \frac{1}{N}\sum_{i=1}^{N}\frac{\partial \mathcal{L}^{(i)}}{\partial \boldsymbol{\theta}}.$$

- Computing the gradient requires summing over *all* of the training examples. This is known as batch training.
- Batch training is impractical if you have a large dataset $N \gg 1$ (e.g. millions of training examples)!

Stochastic Gradient Descent

- Stochastic gradient descent (SGD): update the parameters based on the gradient for a single training example,
  1. Choose i uniformly at random
  2. $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial \boldsymbol{\theta}}$
- Cost of each SGD update is independent of N.
- SGD can make significant progress before even seeing all the data!
- Mathematical justification: if you sample a training example uniformly at random, the stochastic gradient is an unbiased estimate of the batch gradient:

$$\mathbb{E}\left[\frac{\partial \mathcal{L}^{(i)}}{\partial \boldsymbol{\theta}}\right] = \frac{1}{N}\sum_{i=1}^{N} \frac{\partial \mathcal{L}^{(i)}}{\partial \boldsymbol{\theta}} = \frac{\partial \mathcal{J}}{\partial \boldsymbol{\theta}}.$$

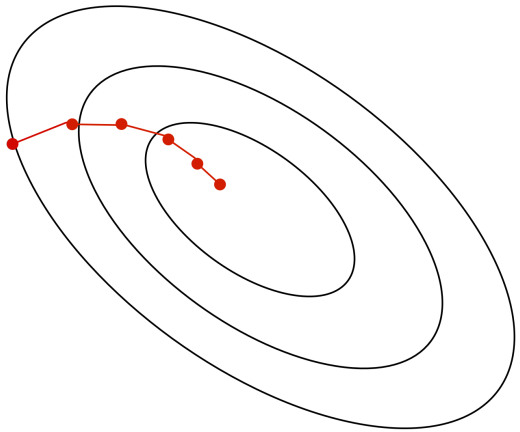
- Problems:
  - Variance in this estimate may be high
  - If we only look at one training example at a time, we can't exploit efficient vectorized operations.

Stochastic Gradient Descent

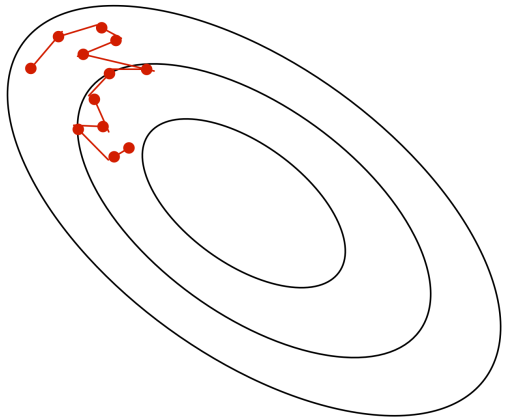
- Compromise approach: compute the gradients on a randomly chosen medium-sized set of training examples $\mathcal{M} \subset \{1, \ldots, N\}$, called a mini-batch.
- Stochastic gradients computed on larger mini-batches have smaller variance. This is similar to bagging.
- The mini-batch size $|\mathcal{M}|$ is a hyperparameter that needs to be set.
  - Too large: takes more computation, i.e. takes more memory to store the activations, and longer to compute each gradient update
  - Too small: can't exploit vectorization, has high variance
  - A reasonable value might be $|\mathcal{M}| = 100$.

Stochastic Gradient Descent

- Batch gradient descent moves directly downhill. SGD takes steps in a noisy direction, but moves downhill on average.



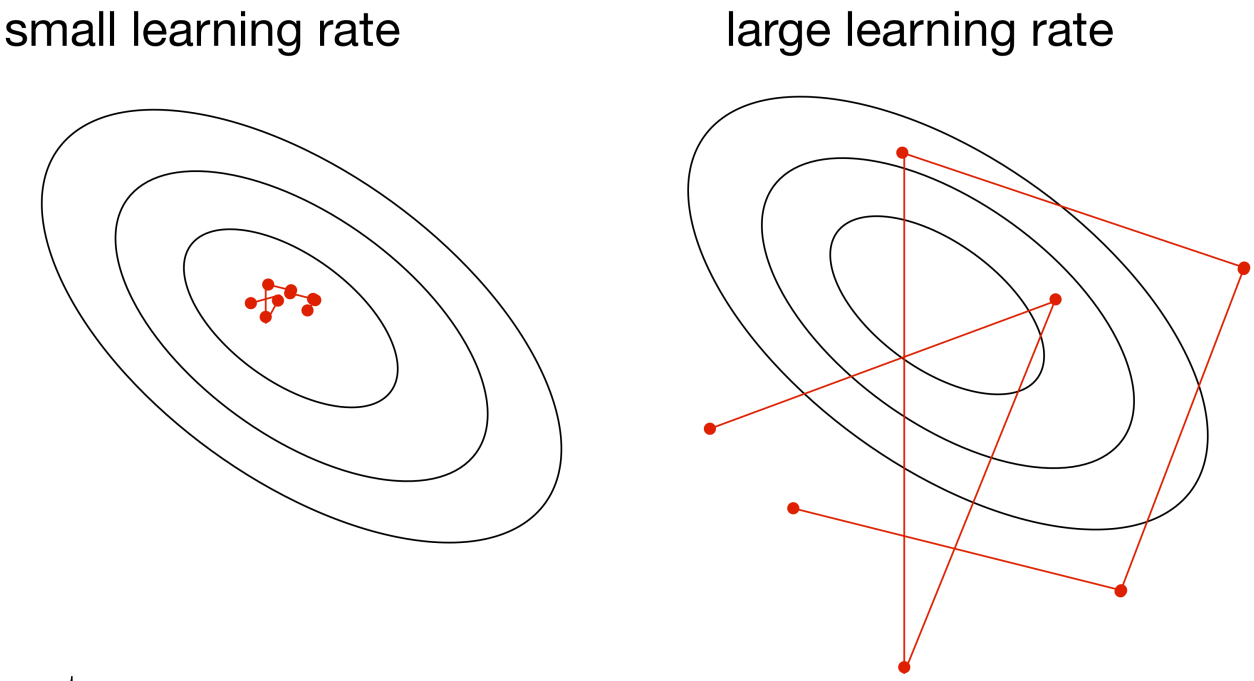
batch gradient descent



stochastic gradient descent

SGD Learning Rate

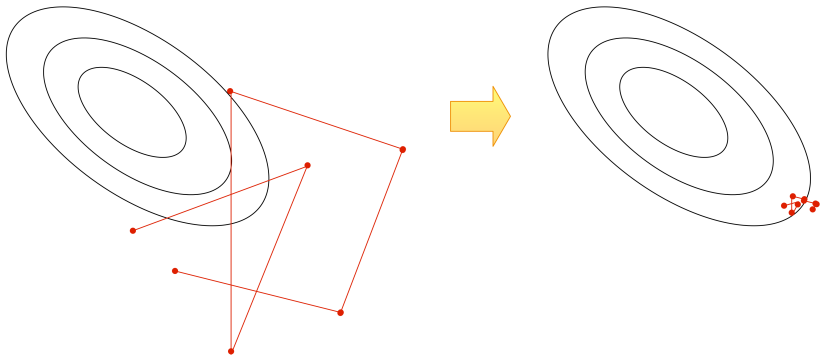
- In stochastic training, the learning rate also influences the fluctuations due to the stochasticity of the gradients.

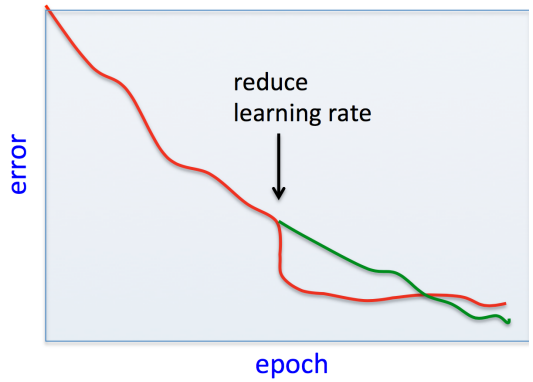


- Typical strategy:
  - Use a large learning rate early in training so you can get close to the optimum
  - Gradually decay the learning rate to reduce the fluctuations

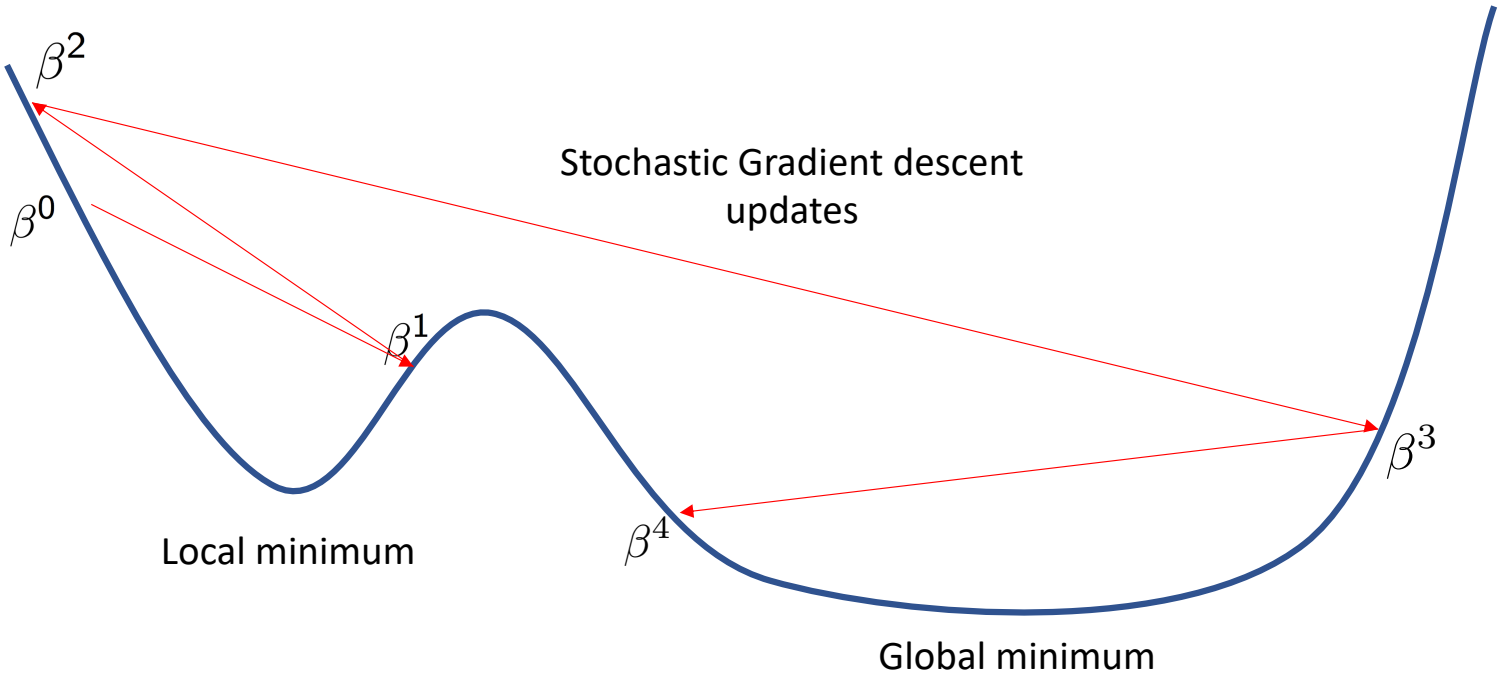
SGD Learning Rate

- Warning: by reducing the learning rate, you reduce the fluctuations, which can appear to make the loss drop suddenly. But this can come at the expense of long-run performance.





SGD and Non-convex optimization



- Stochastic methods have a chance of escaping from bad minima.
- Gradient descent with small step-size converges to first minimum it finds.